Laboratory Exercise 3: Data Structures — Sequences, Mappings & Sets

Student Details	
Student Name: _____	**Student ID:** _____
Duration: 2 Hours (120 Minutes)	**Total Points:** 100 Points

1. Learning Objectives

- **Sequences (Lists vs. Tuples):** Master indexing, slicing (`[start:stop:step]`), and mutability vs. immutability.
- **Iteration & Dictionaries:** Build accumulator loops with `range()` , manipulate key-value dictionaries (`dict`), and perform safe lookups with `.get()` .
- **Set Operations:** Apply mathematical set operations (`&` , `|` , `-` , `issubset()`) to eliminate duplicates and analyze route waypoints.

2. Exercise Structure, Evaluation & Instructor Sign-Off

This 120-minute laboratory session is divided into 5 sequential parts totaling 100 points: **Part 1: Sequence Manipulation: Lists vs. Tuples** (20 mins, 15 pts), **Part 2: Iteration, Range & List Accumulation Patterns** (25 mins, 20 pts), **Part 3: Dictionaries: Key-Value Mapping & Safe Lookups** (25 mins, 20 pts), **Part 4: Practical Modeling: Aircraft Weight & Balance Tracker** (30 mins, 25 pts), and **Part 5: Challenge: Flight Waypoint Route Analyzer with Sets** (20 mins, 20 pts).

Instructor Verification	Details
Score Awarded: _____ / 100 Points	**Instructor Signature:** _____ **Date:** _____ / _____ / 20__
Instructor Comments & Feedback:	_____

Part 1: Sequence Manipulation — Lists vs. Tuples (15 Points)

Task 1.1: Indexing & Slicing Slices (8 Points)

Given the following hourly flight altitude logs recorded in hundreds of feet (Flight Level):

```python
altitude_fl = [180, 240, 310, 350, 390, 390, 370, 220, 100, 30]
```

1. **(2 Points)** Using positive and negative indexing, extract and print the initial takeoff cruise level (`180`) and the final approach level (`30`).
   - **Takeoff Cruise Level (Index 0):** `altitude_fl[____]` = _____
   - **Final Approach Level (Negative Index):** `altitude_fl[____]` = _____
2. **(3 Points)** Using slice notation `[start:stop]` , extract the cruise phase (from index 2 up to, but not including, index 6: `[310, 350, 390, 390]`).
   - **Slice Expression:** `altitude_fl[____:____]`
   - **Resulting Sub-list:** _____
3. **(3 Points)** Using step slicing `[::step]` , extract every second reading from the entire list.
   - **Step Slice Expression:** `altitude_fl[____]`
   - **Resulting List:** _____

Task 1.2: Mutability vs. Immutability (7 Points)

Examine the following code:

```python
runway_list = ["01L", "19R", "09L"]
runway_tuple = ("01L", "19R", "09L")

# 1. Modify the first runway of the list to "01R"
runway_list[0] = "01R"

# 2. Attempt the same on the tuple:
# runway_tuple[0] = "01R"
```

1. Explain in a comment why modifying `runway_list[0]` succeeds, but `runway_tuple[0] = "01R"` throws a `TypeError` .

   Student Explanation:

   _____

   _____
2. Demonstrate how to append `"27R"` to `runway_list` using `.append()` or `+=` .

Code Statement: _____ **Updated** `runway_list` **:**
_____

Part 2: Iteration, Range & List Accumulation (20 Points)

Task 2.1: Fuel Flow Consumption Accumulator (10 Points)

An aircraft engine fuel computer samples instantaneous fuel flow (in gallons per hour) every 15 minutes during a 2-hour flight:

```python
fuel_flows = [120.5, 118.0, 115.2, 112.0, 110.5, 110.0, 108.4, 95.0]
```

Write a `for` loop that:

1. Iterates through the list without using the built-in `sum()` function.
2. Accumulates the sum of all fuel flows.
3. Computes the average fuel flow rate.
4. Finds the minimum and maximum fuel flow recorded using loop comparison statements.

Expected Output:

```text
Total Flow Sum: _______ GPH
Average Fuel Flow: _______ GPH
Max Fuel Flow: _______ GPH
Min Fuel Flow: _______ GPH
```

Student Answer / Program Output:

- **Total Flow Sum:** _____ GPH
- **Average Fuel Flow:** _____ GPH
- **Max Fuel Flow:** _____ GPH
- **Min Fuel Flow:** _____ GPH
-

Task 2.2: Generating Coordinated Altitude Profiles with `range()` (10 Points)

An aircraft performs a stepped climb from $10,000$ ft to $35,000$ ft in increments of $5,000$ ft. Use `range()` inside a loop to generate and append these altitude targets into a new empty list `step_climb = []`. Display the resulting list and its length using `len()`.

Student Answer / Program Output:

- **`range()` Parameters Used:** `range(____, ____, ____)`
- **Generated `step_climb` List:** _____
- **List Length (`len(step_climb)`):** _____ readings

Part 3: Dictionaries — Key-Value Mapping & Safe Lookups (20 Points)

Task 3.1: Airport Fleet Registry (10 Points)

Create a Python dictionary representing an airline fleet at a base hub:

```python
fleet = {
    "TG-401": {"type": "A350-900", "seats": 321, "status": "In-Flight"},
    "TG-402": {"type": "B777-300ER", "seats": 348, "status": "Maintenance"},
    "TG-403": {"type": "B787-9", "seats": 298, "status": "Boarding"},
}
```

Write code to:

1. Change the status of `"TG-402"` from `"Maintenance"` to `"Available"`.
2. Add a new aircraft `"TG-404"`: `{"type": "A321neo", "seats": 192, "status": "Standby"}`.
3. Iterate over the dictionary and print each aircraft callsign and its current status in an aligned column format.

Student Answer / Program Output:

- **Updated Status for TG-402:** _____
- **Added TG-404 Entry:** _____

Task 3.2: Safe Lookup with `.get()` (10 Points)

Write a prompt where the user inputs an aircraft callsign (e.g., `TG-999`). Use `fleet.get(callsign, None)` to look up the aircraft.

- If found, print its aircraft type and seat capacity.
- If not found, print `"[ALERT] Aircraft callsign not found in active fleet manifest."` without raising a `KeyError`.

Student Answer / Program Output:

- **Successful Lookup Output (e.g., TG-401):** _____
- **Non-existent Lookup Output (e.g., TG-999):** _____

Part 4: Practical Modeling: Weight & Balance Manifest (25 Points)

Before departure, flight dispatchers calculate the total payload and center-of-gravity moment for cargo zones.

Given Data Structure:

```python
cargo_manifest = [
    {"zone": "Zone A (Forward)", "weight_kg": 1420.0, "arm_m": 8.5},
    {"zone": "Zone B (Mid-Fwd)", "weight_kg": 2150.0, "arm_m": 14.2},
    {"zone": "Zone C (Mid-Aft)", "weight_kg": 1890.0, "arm_m": 22.0},
    {"zone": "Zone D (Aft)", "weight_kg": 980.0, "arm_m": 28.5}
]
```

Formulas:

- $\text{Zone Moment} = \text{weight_kg} \times \text{arm_m}$
- $\text{Total Cargo Weight} = \sum \text{weight_kg}$
- $\text{Total Cargo Moment} = \sum \text{Zone Moment}$
- $\text{Center of Gravity (CG Arm)} = \frac{\text{Total Cargo Moment}}{\text{Total Cargo Weight}}$

Requirements:

Write a Python program that:

1. Iterates through `cargo_manifest`.
2. Calculates the moment for each zone and prints a tabular loading sheet.
3. Computes the total cargo weight and total cargo moment.
4. Computes the composite CG position in meters.
5. If the resulting CG position is outside the safe envelope ($14.0\ \text{m} \leq \text{CG} \leq 18.0\ \text{m}$), display an `"[UNSAFE CG WARNING]"`; otherwise display `"[SAFE TRIM NORMAL]"`.

Example Run:

```text
Zone Loading Sheet:
- Zone A (Forward): 1420.0 kg | Arm: 8.5 m | Moment: 12070.0 kg-m
- Zone B (Mid-Fwd): 2150.0 kg | Arm: 14.2 m | Moment: 30530.0 kg-m
- Zone C (Mid-Aft): 1890.0 kg | Arm: 22.0 m | Moment: 41580.0 kg-m
- Zone D (Aft): 980.0 kg | Arm: 28.5 m | Moment: 27930.0 kg-m

--- Load Summary ---
Total Cargo Weight: ______ kg
Total Moment: ______ kg-m
Calculated CG Location: ______ meters
Trim Status: ______
```

Student Answer / Program Output:

- **Total Cargo Weight:** _____ kg
- **Total Cargo Moment:** _____ kg-m
- **Calculated CG Location:** _____ meters
- **Trim Status Evaluation:** _____

Part 5: Challenge Task — Flight Waypoint Analyzer with Sets (20 Points)

Air navigation waypoints define flight paths across different Flight Information Regions (FIRs). Two flight plans share overlapping navigation corridors:

```python
flight_A_waypoints = ["BKK", "LIMLA", "BETNO", "SALAX", "DOSEN", "CONOR", "HKG"]
flight_B_waypoints = ["CNX", "BETNO", "SURIN", "DOSEN", "CANTO", "HKG", "TPE"]
```

Requirements:

Using Python **sets** and set methods/operators:

1. Convert both lists into sets: `set_A` and `set_B`.
2. Find and display all **shared waypoints** where both aircraft might cross paths (Intersection: `&`).
3. Find all **unique waypoints used by Flight A but not Flight B** (Difference: `-`).
4. Find the **complete combined network of waypoints** across both flights with all duplicates eliminated (Union: `|`).
5. Check if the set `{"BETNO", "DOSEN"}` is a strict subset (`issubset()`) of both flight plans.

Example Run:

```text
Shared Intersecting Waypoints: {_____}
Waypoints Unique to Flight A: {______}
Total Consolidated Waypoints: _______ waypoints
Waypoint Pair {'BETNO', 'DOSEN'} in Flight A: _______
Waypoint Pair {'BETNO', 'DOSEN'} in Flight B: _______
```

Student Answer / Program Output:

- **Shared Waypoints (Intersection `&`):** _____
- **Unique to Flight A (Difference `-`):** _____
- **Total Consolidated Waypoints (Union `|` Count):** _____ waypoints
- **Subset Check `{"BETNO", "DOSEN"}` in Flight A:** _____
- **Subset Check `{"BETNO", "DOSEN"}` in Flight B:** _____